Data Science e Tecnologie per le Basi di Dati

Antonio Villella

Anno accademico 2026–2027

Indice

1 Data Science e The Big Data Challenge **5**
- 1.1 Introduzione ai Big Data . . . 5
- 1.2 Le Sfide dei Big Data . . . 5
  - 1.2.1 Tecnologia e infrastruttura . . . 5
  - 1.2.2 Gestione e analisi dei dati . . . 6
- 1.3 Data Science . . . 6
- 1.4 Il Processo di Data Science (KDD) . . . 6
  - 1.4.1 Fase 1: Generazione dei dati (Generation) . . . 6

2 Data Warehouse: Introduzione e Architetture **8**
- 2.1 Supporto alle decisioni aziendali . . . 8
  - 2.1.1 Perché mantenere i dati separati? . . . 8
- 2.2 Struttura e analisi dei dati . . . 9
  - 2.2.1 Rappresentazione multidimensionale . . . 9
  - 2.2.2 Rappresentazione relazionale: Il Modello a Stella . . . 9
- 2.3 Architetture per Data Warehouse . . . 10
  - 2.3.1 Elementi costitutivi . . . 10
  - 2.3.2 Tipologie di Server OLAP . . . 10
  - 2.3.3 Architettura a due livelli . . . 11
  - 2.3.4 Architettura a tre livelli . . . 11

3 Progettazione del Data Warehouse **12**
- 3.1 Fattori di rischio . . . 12
- 3.2 Approcci alla progettazione . . . 12
- 3.3 Ciclo di vita della progettazione di un Data Mart . . . 12
  - 3.3.1 Analisi dei requisiti . . . 13
  - 3.3.2 Progettazione concettuale . . . 13
  - 3.3.3 Sparsità dei dati . . . 14
  - 3.3.4 Progettazione logica e ROLAP . . . 14
  - 3.3.5 Progettazione fisica . . . 15

4 Analisi dei Data Warehouse **16**

5 Progettazione fisica e alimentazione dei Data Warehouse **17**

6 Conceptual Schema: Textual Formalism **18**

7 Data Lakes **19**

8 Data Mining Process **20**

9 Data Preparation **21**

10 Data Mining: Association Rules **22**

11 Data Mining: Classificazione **23**

12 Data Mining: Clustering **24**

Introduzione al Corso e Modalità d'Esame

Descrizione dell'Insegnamento

L'insegnamento di Data Science e Tecnologie per le Basi di Dati è un corso obbligatorio per la Laurea Magistrale in Ingegneria Informatica[cite: 1]. L'obiettivo principale del corso è descrivere gli aspetti fondamentali della tecnologia dei sistemi per la gestione delle basi di dati (DBMS) e le metodologie di gestione delle basi di dati destinate all'analisi (comunemente denominate data warehouse)[cite: 1].

Questi sistemi si caratterizzano per la necessità di gestire enormi volumi di dati[cite: 1]. Nel corso vengono esplorate sia le tecniche di analisi tradizionali, come l'analisi OLAP (On Line Analytical Processing), sia tecniche di analisi più complesse basate sul data mining[cite: 1].

Programma e Argomenti Trattati

Il programma del corso si articola in diverse macro-aree fondamentali per la gestione e l'analisi dei dati:

- **Caratteristiche tecnologiche dei DBMS:** vengono studiati la gestione dell'accesso concorrente ai dati, l'affidabilità del sistema, le strutture del livello fisico e l'ottimizzazione dell'accesso ai dati[cite: 1].
- **Basi di dati NoSQL:** introduzione al modello dei dati e ai linguaggi di interrogazione di sistemi non relazionali, con particolare riferimento a MongoDB ed Elastic[cite: 1].
- **Basi di dati distribuite:** studio dell'architettura dei sistemi distribuiti e delle relative tecniche di gestione della replicazione[cite: 1].
- **Data warehouse (DW):** analisi dell'architettura e delle metodologie di progettazione concettuale, logica e fisica[cite: 1]. Vengono inoltre trattate la preparazione dei dati e la formulazione di interrogazioni OLAP tramite il linguaggio SQL[cite: 1].
- **Data science e Machine learning:** panoramica sul processo di data science e approfondimento dei principali algoritmi di apprendimento automatico per la classificazione, il clustering e l'estrazione di regole di associazione[cite: 1].

Struttura e Regole dell'Esame

L'esame si compone di una prova scritta obbligatoria e della valutazione di eventuali homework assegnati durante il semestre (elaborato progettuale individuale)[cite: 1]. Gli homework sono

facoltativi, ma fortemente consigliati[cite: 1]. A discrezione del docente, potrebbe essere richiesta una prova integrativa a conferma del voto ottenuto[cite: 1].

La Prova Scritta

La prova scritta si svolge in aula tramite PC, utilizzando la piattaforma di ateneo, e ha una durata complessiva di 100 minuti[cite: 1]. Durante lo svolgimento non è permesso l'utilizzo di libri, appunti o dispositivi elettronici di alcun tipo[cite: 1].

Il test è composto da domande a risposta aperta e chiusa, oltre a esercizi pratici di progettazione. Le regole di valutazione delle domande prevedono che le risposte errate ai quesiti a risposta chiusa comportino una penalizzazione, mentre le risposte omesse valgano zero[cite: 1].

L*a struttura tipica della prova scritta è la seguente:

- **Domande teoriche (6-8 quesiti):** vertono sui concetti principali del corso, quali aspetti tecnologici dei DBMS, basi di dati distribuite, preparazione dei dati e algoritmi di analisi (massimo 8 punti)[cite: 1].
- **Progettazione fisica (1-3 esercizi):** esercizi sulla creazione di strutture fisiche per basi di dati (massimo 3 punti)[cite: 1].
- **Progettazione Data Warehouse (1-2 esercizi):** modellazione di un DW (massimo 5 punti)[cite: 1].
- **Preparazione dei dati (1 esercizio):** operazioni di data preparation per data warehouse in linguaggio SQL (massimo 5 punti)[cite: 1].
- **Interrogazioni OLAP (3 esercizi):** scrittura di istruzioni SQL per l'accesso e l'analisi dei dati di un data warehouse (massimo 11 punti)[cite: 1].

Homework e Valutazione Finale

Durante il corso verranno proposti quattro homework individuali[cite: 1]. Questi lavori sono mirati a sviluppare il pensiero critico e le capacità di problem solving sugli argomenti trattati a lezione[cite: 1].

Per quanto riguarda il calcolo del voto finale:

- Gli homework possono conferire un massimo di 2 punti aggiuntivi[cite: 1].
- Le relazioni degli homework vengono prese in considerazione per il voto finale esclusivamente se la valutazione della prova scritta è pari o superiore a 18/30[cite: 1].
- La lode viene assegnata nel caso in cui il punteggio complessivo superi il 31/30[cite: 1].

Capitolo 1

Data Science e The Big Data Challenge

1.1 Introduzione ai Big Data

Il concetto di **Big Data** ha molteplici definizioni, essendo un termine che si è evoluto rapidamente nel corso degli anni (spesso accompagnato da un forte *hype* mediatico, come ironicamente illustrato in celebri vignette a tema). Tuttavia, da un punto di vista accademico e tecnologico, possiamo adottare la seguente definizione formale:

Definizione 1.1 (Big Data)**.** I Big Data sono dati la cui **scala, diversità e complessità** richiedono nuove architetture, tecniche, algoritmi e strumenti di analisi (*analytics*) per poterli gestire ed estrarne valore e conoscenza nascosta.

Figura 1.1: L'hype mediatico attorno al concetto di Big Data.

1.2 Le Sfide dei Big Data

L'avvento dei Big Data introduce delle sfide significative che non possono essere affrontate con i sistemi tradizionali. Queste sfide si dividono in due macro-categorie:

1.2.1 Tecnologia e infrastruttura

La gestione di moli di dati così imponenti richiede nuovi paradigmi di programmazione e nuove architetture hardware/software. Il principio fondamentale che guida queste nuove tecnologie è:

"Transfer the processing power to the data"
(Trasferire la potenza di calcolo verso i dati)

Invece di spostare enormi quantità di dati verso i server di calcolo (operazione lenta e costosa in termini di rete), si spostano gli algoritmi e il calcolo direttamente sui nodi dove i dati sono immagazzinati. L'ecosistema di riferimento per questo paradigma è **Apache Hadoop** e **Apache Spark**.

Figura 1.2: L'ecosistema Apache Hadoop è una delle tecnologie fondanti per gestire le infrastrutture Big Data.

1.2.2 Gestione e analisi dei dati

Oltre all'infrastruttura, c'è una nuova e forte enfasi sul "dato" in sé. Raccogliere i dati non basta: è necessario analizzarli per estrarne valore. Questa esigenza ha dato vita a una disciplina specifica: la **Data Science**.

1.3 Data Science

La Data Science è la disciplina che si occupa di *"estrarre significato da grandissime quantità di dati"*. È un campo fortemente multidisciplinare che unisce statistica, machine learning, data mining, visualizzazione dei dati e gestione dei database.

Osservazione 1.1. Il termine *"Data Scientist"* è stato coniato originariamente da D.J. Patil (ex Chief Data Scientist degli Stati Uniti), contribuendo a definire una nuova figura professionale fondamentale nel panorama tecnologico moderno.

Figura 1.3: La natura multidisciplinare della Data Science.

1.4 Il Processo di Data Science (KDD)

Il processo di estrazione della conoscenza dai dati è formalmente noto come **KDD** (*Knowledge Discovery in Databases*).

Il flusso di lavoro della Data Science si articola in una macro-pipeline composta da quattro fasi principali, disposte in sequenza logica:

1. **Generation** (Generazione del dato)
2. **Acquisition** (Acquisizione)
3. **Storage** (Memorizzazione)
4. **Analysis** (Analisi)

Figura 1.4: Rappresentazione del processo KDD (Knowledge Discovery in Databases) e del ciclo di vita di un Data Product.

1.4.1 Fase 1: Generazione dei dati (Generation)

La prima fase del processo KDD riguarda il modo in cui i dati vengono creati nel mondo reale. Possiamo classificare la generazione dei dati in tre tipologie principali:

- **Registrazione passiva (Passive recording):** Produce tipicamente **dati strutturati**. Esempi classici includono le transazioni bancarie e di trading, i registri degli acquisti nei negozi (shopping records) e gli archivi della pubblica amministrazione (government sector archives). L'utente non genera attivamente il dato, ma compie un'azione che viene passivamente registrata dal sistema.
- **Generazione attiva (Active generation):** Produce tipicamente **dati semi-strutturati o non strutturati**. Si tratta principalmente di contenuti generati direttamente dagli utenti (*User-generated content*), come ad esempio i post, le foto e i video caricati sui social network.

- **Produzione automatica (Automatic production):** Riguarda dati generati in modo continuo e automatico da dispositivi. Questi dati sono spesso *location-aware* (geolocalizzati), *context-dependent* (dipendenti dal contesto) e generati da dispositivi altamente mobili (come smartphone, sensori IoT, smartwatch).

Capitolo 2

Data Warehouse: Introduzione e Architetture

2.1 Supporto alle decisioni aziendali

La maggior parte delle aziende, non solo di grandi dimensioni ma oggi anche quelle più piccole, è dotata di un proprio sistema informatico (database operazionale). Questi sistemi informativi sono progettati per garantire e gestire l'operatività quotidiana (ad esempio, le transazioni di un'assicurazione o le vendite in un negozio) e costituiscono una potenziale miniera di informazioni utili.

Tuttavia, esiste una netta distinzione tra i sistemi per la gestione ordinaria e i sistemi decisionali. Secondo W. H. Inmon (1992), i **sistemi per il supporto alle decisioni (DSS - Decision Support Systems)** permettono di analizzare lo stato dell'azienda per *prendere decisioni rapide e migliori.*

2.1.1 Perché mantenere i dati separati?

Per fare analisi avanzate, non si interroga direttamente il database operativo, ma si utilizza un **Data Warehouse (DW)**, un ambiente separato. Questa separazione è motivata da due ragioni principali:

- **Prestazioni:** Le query di analisi (ricerche complesse) sono onerose e ridurrebbero drasticamente le prestazioni delle transazioni operative quotidiane. Inoltre, i due sistemi richiedono metodi di accesso ai dati diversi a livello fisico.
- **Gestione dei dati:** I database operazionali spesso non mantengono lo storico (informazioni mancanti nel tempo). Un DW permette il consolidamento dei dati storici e risolve problemi di qualità, eliminando le inconsistenze che possono essere presenti nei sistemi operativi.

Osservazione 2.1 (Il Data Warehouse è solo una vista?). Durante la lezione è emersa una domanda: *"Il Data Warehouse si può definire come una vista elaborata della base di dati già esistente?"*
Concettualmente, l'intuizione è corretta: può essere visto come una prima vista molto elaborata su cui poi se ne creano altre per ottimizzare le query. Tuttavia, fisicamente e operativamente, **è quasi sempre un DB separato**. Questo perché il DW non si limita a riproporre i dati aziendali, ma spesso integra dati provenienti da sorgenti esterne diverse, richiedendo un processo di integrazione molto più articolato rispetto a una semplice 'VIEW' in SQL.

2.2 Struttura e analisi dei dati

2.2.1 Rappresentazione multidimensionale

Nel Data Warehouse, l'informazione viene modellata in ottica analitica. I dati vengono rappresentati concettualmente come un **(iper)cubo** con tre o più dimensioni. Gli elementi fondamentali di questo modello sono:

- **Dimensioni (Assi dimensionali):** Descrivono il contesto dell'analisi (es. tempo, prodotto, negozio).
- **Misure:** Sono gli elementi numerici individuati all'intersezione delle dimensioni, sui quali si eseguono effettivamente i calcoli e le analisi (es. quantità venduta, importo).

Figura 2.1: Rappresentazione logica del dato multidimensionale nel Data Warehouse.

2.2.2 Rappresentazione relazionale: Il Modello a Stella

Se vogliamo tradurre questo ipercubo logico in un modello relazionale fisico (su un RDBMS), utilizziamo il cosiddetto **modello a stella**.

Definizione 2.1 (Modello a Stella)**.** Modello di progettazione fisica in cui:

- Le **misure numeriche** (i fatti) vengono memorizzate all'interno di una tabella centrale chiamata *Tabella dei Fatti*.
- Le **dimensioni di analisi** sono memorizzate in tabelle esterne collegate alla tabella centrale, e contengono gli attributi descrittivi che caratterizzano i fatti.

Facendo un parallelo con la progettazione relazionale classica studiata nel corso di Basi di Dati: avremo un'entità per il Negozio, una per il Tempo e una per il Prodotto. L'evento della "Vendita" è caratterizzato dalle chiavi esterne verso queste dimensioni e dai rispettivi valori numerici (quantità e importo). Sebbene si usino le stesse strutture (tabelle), nel DW queste tabelle subiscono un processo di **denormalizzazione** per semplificare e velocizzare le operazioni di lettura e analisi. "Più che una differenza nel *cosa*, c'è una differenza nel *perché* utilizzo A e *perché* utilizzo B", sottolinea il professore.

Figura 2.2: Esempio di modello a stella per l'analisi delle vendite in una catena di supermercati.

Esempio 2.1 (Dimensione del Data Warehouse)**.** Per comprendere il volume di dati gestito, consideriamo un esempio per l'analisi delle vendite di una catena di supermercati:

- Dimensione tempo: 2 anni = 730 giorni
- Dimensione negozio: 300 negozi
- Dimensione prodotto: 30.000 prodotti a catalogo. Supponiamo che 3.000 prodotti vengano venduti ogni giorno in ogni negozio.

Il numero di righe nella tabella dei fatti per questo intervallo temporale sarà:

$$730 \times 300 \times 3000 = 657.000.000 \text{ (657 milioni di righe)}$$

2.3 Architetture per Data Warehouse

Per evitare colli di bottiglia, si evita tassativamente un'architettura a livello singolo (un unico DB per operazioni e analisi). Si adottano invece architetture a due o più livelli che separano i dati in ingresso dal carico analitico, rendendo il sistema fortemente scalabile.

2.3.1 Elementi costitutivi

Il processo che porta i dati dalle **sorgenti (operative ed esterne)** al supporto decisionale è gestito da diversi componenti:

1. **Strumenti ETL (Extraction, Transformation, Loading):** Procedure responsabili dell'estrazione dei dati dalle sorgenti, della loro pulizia, rielaborazione e integrazione (Transformation), e infine del caricamento (Loading) nel Data Warehouse.
2. **Data Warehouse / Data Marts:** I repository dove i dati analitici vengono fisicamente memorizzati.
3. **OLAP Servers:** Motori (Online Analytical Processing) che permettono di effettuare in modo efficace le query sul Data Warehouse.
4. **Strumenti di Visualizzazione e Analisi:** Strumenti finali utilizzati dall'utente. La visualizzazione è cruciale: un grafico offre un'intuizione molto più immediata rispetto a una tabella numerica. I grafici devono inoltre essere *navigabili* (es. permettere operazioni di approfondimento come il *drill-down* per cercare motivazioni dietro ai dati).

Definizione 2.2 (Metadati)**.** I metadati sono "dati sui dati", ovvero informazioni essenziali per il funzionamento e l'interrogazione del Data Warehouse. Si dividono in:

- **Metadati per ETL:** Descrivono i dati sorgente (dove recuperarli, in che formato, qual è il periodo di riferimento, le variazioni storiche) e le trasformazioni necessarie. (Esiste uno standard OMG chiamato *CWMI - Common Warehouse Metadata Initiative* per l'interscambio di questi metadati).
- **Metadati per la gestione dei dati:** Descrivono lo schema logico del Data Warehouse e la struttura dei dati derivati (es. viste materializzate).
- **Metadati per la gestione delle query:** Contengono il codice SQL, i piani di esecuzione e statistiche di monitoraggio (uso di RAM e CPU).

2.3.2 Tipologie di Server OLAP

Esistono diverse tecnologie per i server OLAP a supporto del DW:

- **ROLAP (Relational OLAP):** Si basa su DBMS relazionali estesi. Offre una rappresentazione compatta per dati sparsi (matrici con molti zeri) e possiede estensioni SQL e metodi di accesso speciali per gli aggregati.
- **MOLAP (Multidimensional OLAP):** I dati sono memorizzati direttamente in un formato matriciale multidimensionale proprietario. Poiché i dati reali sono molto "sparsi", questa architettura richiede pesanti tecniche di compressione. Offre primitive OLAP speciali molto efficienti.
- **HOLAP (Hybrid OLAP):** Un approccio ibrido che sfrutta i vantaggi di entrambe le tecnologie precedenti.

2.3.3 Architettura a due livelli

In un'architettura a due livelli abbiamo: il livello delle sorgenti e il livello del Data Warehouse. In questo modello, le operazioni ETL avvengono tipicamente *"al volo" (on the fly)* durante il passaggio dalle sorgenti al DW. Questa architettura permette:

- Il disaccoppiamento dalle sorgenti e la gestione di dati esterni all'OLTP.
- Una modellazione fisica dei dati mirata esclusivamente al carico analitico (OLAP).
- La facilità di gestire granularità temporali differenti tra i dati operativi e quelli analitici.

Figura 2.3: Architettura Data Warehouse a due livelli.

2.3.4 Architettura a tre livelli

L'architettura a tre livelli introduce un elemento intermedio fondamentale per gestire flussi dati più complessi e disaccoppiare ulteriormente il caricamento dall'analisi:

1. Livello delle sorgenti.
2. **Livello di alimentazione (Staging Area):** Un'area di transito o "parcheggio" temporaneo in cui i dati estratti dalle sorgenti vengono depositati e processati (trasformati, puliti e integrati) dagli strumenti ETL prima del caricamento definitivo.
3. Livello del Data Warehouse.

Figura 2.4: Architettura Data Warehouse a tre livelli con Staging Area.

Capitolo 3

Progettazione del Data Warehouse

La progettazione di un data warehouse per il supporto strategico aziendale richiede un'architettura che mantenga separati i sistemi di supporto decisionale rispetto ai tradizionali DBMS utilizzati per la gestione ordinaria (sistemi OLTP). L'obiettivo non è riversare tutti i dati aziendali nel data warehouse, ma estrarre dalle sorgenti unicamente i dati utili per le analisi di interesse, creando una visione d'insieme uniforme.

3.1 Fattori di rischio

La realizzazione di un data warehouse è un progetto complesso che espone l'azienda a diversi fattori di rischio[cite: 1]:

- **Aspettative elevate degli utenti:** il data warehouse viene spesso visto, erroneamente, come la soluzione magica a tutti i problemi aziendali.
- **Qualità dei dati e dei processi OLTP di partenza:** la presenza di dati incompleti o inaffidabili, unita a processi aziendali non integrati o non ottimizzati, si ripercuote sulla qualità del data warehouse.
- **Gestione "politica" del progetto:** è fondamentale la collaborazione con i "detentori" delle informazioni nei vari reparti e l'accettazione del nuovo sistema da parte degli utenti finali.

3.2 Approcci alla progettazione

Esistono due filosofie principali per la progettazione di un data warehouse[cite: 1]:

- **Approccio top-down (Inmon):** Prevede la realizzazione di un data warehouse centralizzato che fornisca una visione globale e completa di tutti i dati aziendali. Questo approccio comporta costi significativi, tempi di realizzazione lunghi e una fase di analisi e progettazione estremamente complessa.
- **Approccio bottom-up (Kimball):** Prevede una realizzazione incrementale. Si parte creando singoli *data mart* definiti su settori o processi aziendali specifici (es. vendite, acquisti). Ha il vantaggio di mantenere contenuti sia i costi che i tempi di consegna.

3.3 Ciclo di vita della progettazione di un Data Mart

La progettazione di un data mart si articola in diverse fasi metodologiche[cite: 1]:

Figura 3.1: Fasi della progettazione di un data mart.

3.3.1 Analisi dei requisiti

L'analisi dei requisiti è il punto di partenza e serve a raccogliere[cite: 1]:

- Le esigenze di analisi dei dati che il data mart dovrà soddisfare.
- I vincoli realizzativi imposti dai sistemi informativi esistenti (scegliendo se creare data mart dipendenti o indipendenti).
- La pianificazione del *deployment*, ovvero le fasi di avviamento e di formazione degli utenti.

3.3.2 Progettazione concettuale

A differenza delle basi di dati relazionali, nel data warehouse non esiste un formalismo di modellazione concettuale comunemente accettato[cite: 1]. Il tradizionale Modello Entità-Relazione (ER) non è adatto a rappresentare nativamente i concetti multidimensionali.

Osservazione 3.1. Se si provasse a rappresentare uno schema multidimensionale in forma normalizzata (come nell'ER), si creerebbe una struttura in cui tutte le dipendenze funzionali sono frammentate in tabelle separate. Durante le interrogazioni (analisi), questo richiederebbe l'esecuzione di un numero elevatissimo di `JOIN`, degradando in modo inaccettabile le prestazioni.

Per questo motivo, si adotta il **Dimensional Fact Model (DFM)** (Golfarelli, Rizzi). Questo modello definisce, per uno specifico fatto aziendale (es. la vendita), uno schema concettuale basato su:

- **Fatto:** l'evento di interesse da analizzare.
- **Misure:** gli attributi quantitativi del fatto (es. quantità venduta, incasso).
- **Dimensioni:** le prospettive di analisi (es. prodotto, negozio, data).
- **Gerarchie:** i livelli di aggregazione lungo le dimensioni.

Figura 3.2: Esempio di Dimensional Fact Model per le vendite.

DFM: Costrutti avanzati

Il DFM permette di gestire situazioni reali più complesse tramite costrutti avanzati[cite: 1]:

- **Dimensione opzionale, non-additività e convergenza:** Gestione di attributi non sempre presenti, misure non sommabili (es. prezzi unitari) e percorsi gerarchici che si ricongiungono.
- **Gerarchia condivisa e Ruolo:** Quando una dimensione gioca più ruoli nello stesso fatto (es. in una chiamata telefonica abbiamo "distretto chiamante" e "distretto chiamato", che attingono alla stessa gerarchia spaziale).
- **Arco multiplo (Relazioni molti-a-molti):** Nella maggior parte dei casi, lungo una gerarchia le relazioni sono 1 a N (dipendenze funzionali). Tuttavia, esistono contesti in cui si ha una relazione N a N.

Esempio 3.1. Esempio di Arco Multiplo: Il professore evidenzia il caso di un Fatto relativo alla vendita di libri. Un libro può essere scritto da *più* autori. In questo caso non c'è una semplice dipendenza funzionale Libro $\rightarrow$ Autore, ma una relazione molti-a-molti che va esplicitata chiaramente nel modello concettuale tramite un arco multiplo. Un altro classico esempio medico è il Fatto RICOVERO: un ricovero può avere associato a sé *molteplici* diagnosi (gruppo di diagnosi).

Figura 3.3: Costrutti avanzati: rappresentazione dell'arco multiplo nel DFM.

3.3.3 Sparsità dei dati

Un fenomeno da tenere in considerazione è la sparsità del cubo multidimensionale. La sparsità si riduce al crescere del livello di aggregazione dei dati[cite: 1]. Un'alta sparsità può ridurre l'affidabilità delle stime di cardinalità quando si estraggono dati molto aggregati.

3.3.4 Progettazione logica e ROLAP

Terminata la progettazione concettuale, si procede alla progettazione logica relazionale (**ROLAP** - Relational OLAP). Come sottolineato dal professore, il modello multidimensionale è ottimo per ragionare e visualizzare, ma per memorizzare i dati su disco si utilizza il modello relazionale *fortemente denormalizzato*.

Gli **input** della progettazione logica sono[cite: 1]:

- Schema di fatto concettuale.
- Carico di lavoro (stimato in base a quante query verranno fatte, con quale frequenza e quali report/dashboard dovranno essere alimentati).
- Volume dei dati.
- Vincoli di sistema.

L'**output** è lo schema logico relazionale del data warehouse[cite: 1].

La progettazione logica dei data warehouse si basa su principi opposti rispetto alle basi di dati tradizionali: si accetta deliberatamente la **ridondanza dei dati** e si applica la **denormalizzazione delle tabelle** per ottimizzare i tempi di lettura[cite: 1].

Schema a stella (Star Schema)

Nello schema a stella si ha[cite: 1]:

- Una singola tabella centrale dei Fatti.
- **Una sola tabella per ogni dimensione**, fortemente denormalizzata (es. la tabella Negozio conterrà colonne per Negozio, Città e Stato ridondando i dati geografici).
- L'uso di chiavi primarie generate artificialmente (*surrogate keys*) per le dimensioni.

Figura 3.4: Schema a stella (Star Schema).

Schema a fiocco di neve (Snowflake Schema)

Nello schema snowflake si separano alcune dipendenze funzionali, normalizzando parzialmente le dimensioni. I dati di una dimensione vengono frazionati in più tabelle creando delle ramificazioni[cite: 1] (es. la tabella Prodotti punta a una tabella separata Tipi).

Osservazione 3.2. **Quale scegliere?** Lo schema snowflake è normalmente **sconsigliato**. La normalizzazione permette di risparmiare spazio su disco, ma in ambito data warehouse questo beneficio è del tutto trascurabile, mentre il costo computazionale per eseguire i numerosi `JOIN` addizionali penalizza pesantemente le prestazioni delle query analitiche[cite: 1].

Implementazione logica degli archi multipli

Per tradurre un arco multiplo (es. ordini → più città di spedizione/competenza o fatturazioni) dallo schema concettuale a quello logico, esistono due strade[cite: 1]:

- **Bridge Table:** Si esplicita la relazione N:M creando una tabella ponte (bridge) che contiene anche eventuali pesi associati alla relazione.
- **Push down:** Si integra la risoluzione del molti-a-molti spingendo (push down) le informazioni direttamente nella tabella dei fatti, pesando le misure.

Tuttavia, il push down presenta grosse problematiche: rende difficile eseguire interrogazioni d'impatto, richiede di pre-calcolare i pesi durante la fase di alimentazione (ETL), rende ardue le modifiche successive ai dati storicizzati e introduce una forte ridondanza direttamente nella tabella dei fatti[cite: 1].

3.3.5 Progettazione fisica

La **progettazione fisica** è l'ultimo step e decide esattamente *come* i dati devono essere memorizzati fisicamente sui dischi e come verranno effettuate le operazioni di accesso (creazione di indici, partizionamento, ecc.)[cite: 1]. Come ricordato dal professore, siccome le query nel data warehouse estraggono enormi volumi di dati, questa fase è di vitale importanza per garantire che il sistema risponda in tempi utili.

Capitolo 4

Analisi dei Data Warehouse

Capitolo 5

Progettazione fisica e alimentazione dei Data Warehouse

Capitolo 6

Conceptual Schema: Textual Formalism

Capitolo 7

Data Lakes

Capitolo 8

Data Mining Process

Capitolo 9

Data Preparation

Capitolo 10

Data Mining: Association Rules

Capitolo 11

Data Mining: Classificazione

Capitolo 12

Data Mining: Clustering